PRACTICAL : 02

Aim:

Implement breadth-first search, depth-first search and uniform-cost search on graph or maze problems; compare search cost and path quality.

Theory:

A simple reflex agent selects an action based only on the current state of the environment. It uses predefined condition-action rules. In this program, the agent observes whether a location is dirty or clean. If it is dirty, the agent cleans it; otherwise, it moves to the next location.

Program:

```
print("Welcome to Graph Search Algorithms")


# Graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': ['G'],
    'E': ['G'],
    'F': ['G'],
    'G': []
}


# Cost of each path
```

```
cost = {
   'A': {'B': 2, 'C': 5},
   'B': {'D': 4, 'E': 1},
   'C': {'F': 2},
   'D': {'G': 2},
   'E': {'G': 5},
   'F': {'G': 1}
}

# Breadth First Search
def bfs(start, goal):
   queue = [[start]]

   while queue:
      path = queue.pop(0)

      if path[-1] == goal:
         return path

      for node in graph[path[-1]]:
         queue.append(path + [node])

# Depth First Search
def dfs(start, goal):
   stack = [[start]]
```

```
    while stack:
        path = stack.pop()

        if path[-1] == goal:
            return path

        for node in graph[path[-1]]:
            stack.append(path + [node])

# Uniform Cost Search
def ucs(start, goal):
    queue = [(0, [start])]

    while queue:
        queue.sort()
        total, path = queue.pop(0)

        if path[-1] == goal:
            return path, total

        for node in graph[path[-1]]:
            new_cost = total + cost[path[-1]][node]
            queue.append((new_cost, path + [node]))

# Find paths
bfs_path = bfs('A', 'G')
```

```
dfs_path = dfs('A', 'G')
ucs_path, ucs_cost = ucs('A', 'G')

# Calculate path cost
def path_cost(path):
    total = 0

    for i in range(len(path) - 1):
        total = total + cost[path[i]][path[i + 1]]

    return total

# Display results
print("\nBFS Path:", " -> ".join(bfs_path))
print("BFS Cost:", path_cost(bfs_path))

print("\nDFS Path:", " -> ".join(dfs_path))
print("DFS Cost:", path_cost(dfs_path))

print("\nUCS Path:", " -> ".join(ucs_path))
print("UCS Cost:", ucs_cost)

print("\nComparison:")
print("BFS Path Quality: Short path")
print("DFS Path Quality: Depends on search order")
print("UCS Path Quality: Lowest cost path")
```

Output:

```
Welcome to Graph Search Algorithms

BFS Path: A -> B -> D -> G
BFS Cost: 8

DFS Path: A -> C -> F -> G
DFS Cost: 8

UCS Path: A -> B -> D -> G
UCS Cost: 8

Comparison:
BFS Path Quality: Short path
DFS Path Quality: Depends on search order
UCS Path Quality: Lowest cost path
```

Conclusion:

Thus, BFS, DFS, and UCS were successfully implemented. BFS finds a path based on the minimum number of edges, DFS explores paths depth-first, while UCS finds a path based on the minimum total path cost. The comparison demonstrates the difference between path length and path cost